ASSIGNMENT 9

BACKEND, DATABASE, AND AI INTEGRATION

BY: JHANRIC HANAMICHI C PABLO

UNDERSTANDING BACKEND SYSTEMS

THE BACKEND SYSTEM

The server-side component responsible for data storage, processing, and business logic. In AI, it hosts models, manages datasets, and exposes API endpoints for the frontend.

THE DATABASE

An organized collection of structured information. Critical for AI to store training data, model parameters, and inference results with integrity and speed.

AI EXAMPLE: NETFLIX

Uses databases to store user viewing history and movie metadata. The backend AI processes this to generate personalized recommendations.

FRONTEND VS BACKEND

FEATURE	FRONTEND	BACKEND
Visibility	User-facing UI	Server-side Logic
Role in AI	Presents results	Processes models

DATABASE VS API

FEATURE	DATABASE	API
Purpose	Store & Manage	Communicate
Interaction	SQL Queries	HTTP Requests

DATA MANAGEMENT IN AI

RESTFUL API & CRUD

A RESTful API uses standard HTTP methods to interact with AI resources. CRUD represents the four essential operations:

- **CREATE (POST)** Add new data records
- **READ (GET)** Retrieve existing data
- **UPDATE (PUT)** Modify existing records
- **DELETE (DELETE)** Remove data records

WHY IT MATTERS

- Model Performance
- Scalability
- Reproducibility
- Security & Privacy

API & DATABASE SYNERGY

The API acts as the gateway between the user and the data stored in the database.

FRONTEND REQUEST

↓

API LOGIC / AI MODEL

↓

DATABASE (CRUD)

↓

FORMATTED RESPONSE

"Data management ensures AI models are trained on relevant, clean, and diverse datasets for maximum accuracy."

FLASK + AI INTEGRATION

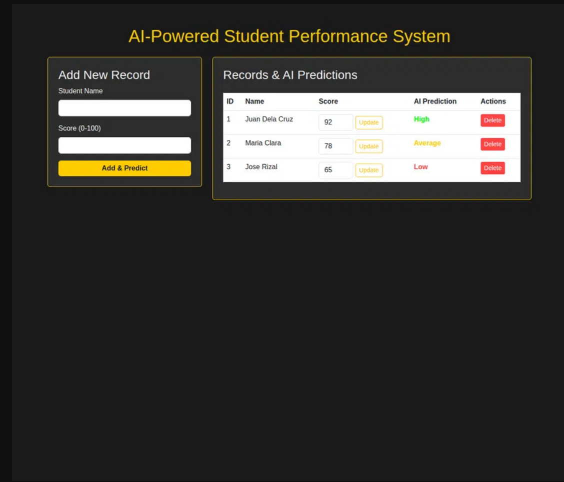
BACKEND LOGIC (FLASK + SQLITE)

```
@app.route('/add', methods=['POST'])
def add_record():
    name = request.form['name']
    score = int(request.form['score'])
    prediction = get_ai_prediction(score)

    conn = get_db_connection()
    conn.execute('INSERT INTO records (name, score, prediction) VALUES (?, ?, ?)', (name, score, prediction))
    conn.commit()
    conn.close()
    return redirect(url_for('index'))

def get_ai_prediction(score):
    if score >= 85: return "High"
    elif score >= 75: return "Average"
    else: return "Low"
```

FUNCTIONAL OUTPUT



AI LOGIC PARAMETERS

- ✓ Score **≥ 85**: High Performance
- ✓ Score **75-84**: Average
- ✓ Score **< 75**: Low